

Aufgabe 1: Metriken

1. Konfigurationen

Wir haben neben der Basisversion vier weitere Varianten des Programms `primesum.c` erstellt und getestet. Die Varianten unterscheiden sich durch den verwendeten Datentyp (`int64_t` vs. `__int128`), die Modulo-Berechnung und die Compiler-Optimierung.

- **v1_base**: `int64_t`, Modulo $10^9 + 7$, **-O0** (Basisversion).
- **v2_opt**: `int64_t`, Modulo $10^9 + 7$, **-O3**.
- **v3_fastmod**: `int64_t`, Modulo 16^8 (bzw. $0x100000000$), **-O3**.
- **v4_int128**: `__int128`, Modulo $10^9 + 7$, **-O3**.
- **v5_no_opt**: `int64_t`, Modulo 16^8 , **-O0**.

2. Hypothesen (Erwartungen)

Vor der Messung hatten wir folgende Erwartungen:

1. **Optimierung**: Wir erwarten, dass `v2_opt` deutlich schneller ist als `v1_base`, da **-O3** effizienteren Maschinencode erzeugt.
2. **Modulo**: Wir erwarten, dass `v3_fastmod` die schnellste Variante sein wird. Die Modulo-Operation mit einer Zweierpotenz (16^8) kann durch eine schnelle bitweise UND-Operation (`&`) ersetzt werden, was viel effizienter ist als eine Division.
3. **Datentyp**: Wir erwarten, dass `v4_int128` am langsamsten ist, da unser Prozessor (64-Bit) 128-Bit-Zahlen emulieren muss und der Speicherbedarf für den Cache doppelt so hoch ist.

3. Ergebnisse & Auswertung

Die Messungen wurden mit `hyperfine` durchgeführt. Hier sind die Ergebnisse:

```
Benchmark 1: ./v1_base
Time (mean ± σ):      814.7 ms ± 29.5 ms    [User: 778.3 ms, System: 29.4 ms]
Range (min ... max):  786.9 ms ... 857.3 ms  10 runs

Benchmark 2: ./v2_opt
Time (mean ± σ):      616.6 ms ±  9.5 ms    [User: 580.9 ms, System: 28.1 ms]
Range (min ... max):  599.4 ms ... 629.1 ms  10 runs

Benchmark 3: ./v3_fastmod
Time (mean ± σ):      620.3 ms ± 16.7 ms    [User: 579.8 ms, System: 30.5 ms]
Range (min ... max):  600.1 ms ... 648.4 ms  10 runs

Benchmark 4: ./v4_int128
Time (mean ± σ):      910.0 ms ± 17.6 ms    [User: 849.0 ms, System: 52.5 ms]
Range (min ... max):  884.2 ms ... 944.0 ms  10 runs

Benchmark 5: ./v5_fastmod_no_opt
Time (mean ± σ):      816.1 ms ± 39.5 ms    [User: 775.7 ms, System: 32.3 ms]
Range (min ... max):  793.9 ms ... 913.0 ms  10 runs
```

- **v2_opt (616 ms)** war die schnellste Version (1.32x schneller als Basis).
- **v3_fastmod (620 ms)** war überraschenderweise **nicht schneller** als `v2_opt`.
- **v4_int128 (910 ms)** war die langsamste Version (sogar langsamer als die Basisversion ohne Optimierung).

```
Summary
./v2_opt ran
 1.01 ± 0.03 times faster than ./v3_fastmod
 1.32 ± 0.05 times faster than ./v1_base
 1.32 ± 0.07 times faster than ./v5_fastmod_no_opt
 1.48 ± 0.04 times faster than ./v4_int128
```

Analyse:

1. **Compiler-Optimierung (-O3):** Hat wie erwartet einen großen Einfluss und bringt etwa 30% Leistungssteigerung.
2. **Memory Bound (Speicherlimitierung):** Dass v3_fastmod nicht schneller war, zeigt, dass das Programm nicht durch die Rechenleistung der CPU (ALU), sondern durch den **Speicherzugriff** begrenzt ist. Die CPU wartet auf Daten aus dem RAM, daher bringt eine schnellere Division keinen Vorteil ("Latency Hiding").
3. **int128 Problem:** Die Variante v4_int128 ist langsam, weil sie die **räumliche Lokalität (Spatial Locality)** verschlechtert. Da ein int128 doppelt so groß ist wie int64, passen nur halb so viele Elemente in eine Cache-Line. Das führt zu mehr Cache-Misses und höherem Speicherverkehr.

Aufgabe 1.2: Energie und Kostenvergleich

Um die Effizienz auf unterschiedlicher Hardware zu bewerten, haben wir das Programm zusätzlich auf einem älteren Laptop ausgeführt und die Ergebnisse verglichen.

1. Geräteparameter und Messung

Gerät A (Referenz):

- **Modell:** Apple MacBook (Apple Silicon).
- **Leistungsaufnahme (P_A):** ca. 20 Watt.
- **Gemessene Zeit (t_A):** 0.910 s (Variante v4_int128).

Gerät B (Vergleichsgerät):

- **Modell:** Acer Aspire 3 (A315).
- **Prozessor:** Intel Core i3-6006U (2.0 GHz, Dual Core).
- **Leistungsaufnahme (P_B):** ca. 30 Watt.
- **Gemessene Zeit (t_B):** 2.942 s.

Beobachtung: Das ältere Gerät benötigt für die gleiche Aufgabe mehr als die dreifache Zeit (3.2x).

2. Energieverbrauch ($E = P \cdot t$)

Gerät A (Mac):

$$E_A = 20 \text{ W} \cdot 0.910 \text{ s} \approx 18.2 \text{ Joule}$$

Gerät B (Acer):

$$E_B = 30 \text{ W} \cdot 2.942 \text{ s} \approx 88.3 \text{ Joule}$$

Fazit: Der Acer Aspire 3 verbraucht für die gleiche Rechenoperation fast **5-mal mehr Energie** (88.3 J vs 18.2 J) als das moderne System.

3. Stromkosten

Wir berechnen die Kosten basierend auf dem aktuellen Börsenstrompreis (EPEX SPOT Day-Ahead, DE-LU, ca. 110 €/MWh). Preis pro kWh: 0.11 €.

Kosten Gerät A:

$$K_A = \frac{18.2}{3.6 \cdot 10^6} \text{ kWh} \cdot 0.11 \left(\frac{\text{€}}{\text{kWh}} \right) \approx 5.56 \cdot 10^{-7} \text{ €}$$

Kosten Gerät B:

$$K_B = \frac{88.3}{3.6 \cdot 10^6} \text{ kWh} \cdot 0.11 \left(\frac{\text{€}}{\text{kWh}} \right) \approx 2.70 \cdot 10^{-6} \text{ €}$$

Fazit: Das ältere System ist deutlich ineffizienter und verbraucht fast 5-mal mehr Energie für dieselbe Berechnung.

4. Begründung

Die drastischen Unterschiede lassen sich durch die Speicherhierarchie erklären:

- **CPU Stalls durch Latenz:** Das Programm ist speicherlastig (Memory Bound). Das ältere System hat eine langsamere Anbindung an den Hauptspeicher. Die CPU muss häufiger und länger auf Daten warten ("Stall"), bevor sie weiterrechnen kann. Während dieser Wartezeit wird Energie verbraucht, ohne dass Arbeit verrichtet wird.

- **Räumliche Lokalität:** Die Verwendung von `int128` halbiert die Anzahl der Elemente, die in einen Cache-Block passen. Das erzwingt häufigeres Nachladen aus dem langsamen Hauptspeicher, was das ältere Speichersystem stärker belastet als das moderne.

5. Verfälschende Faktoren

Warum weicht die Theorie von der Praxis ab?

- **Verdrängung im Cache:** Auf dem Laptop laufen nebenher andere Programme (Betriebssystem, Browser etc.). Diese greifen ebenfalls auf den Speicher zu und verdrängen Teile meiner Daten aus dem Cache. Beim nächsten Zugriff müssen diese Daten wieder zeitaufwendig aus dem RAM geholt werden.
- **Cache-Status (Cold vs. Warm):** Die erste Ausführung des Programms ist immer langsamer, da der Cache zunächst leer ("kalt") ist und die Daten erst aus dem Hauptspeicher geladen werden müssen.